



50 Conceptos Básicos para Aprender a Programar

De no saber nada a pensar como programador. Sin relleno. Ejemplos en JavaScript.

Curso práctico · 50 lecciones · Glosario · Chuleta de sintaxis



50 Conceptos Básicos para Aprender a Programar

*El curso directo, sin relleno, para pasar de cero a entender cómo piensa un programador. Ejemplos en **JavaScript**, el lenguaje que corre en todos los navegadores del mundo.*

¿Para quién es este curso?

- Para quien **nunca ha escrito una línea de código** y quiere empezar bien.
- Para quien ya tocó algo de programación pero tiene **huecos en los fundamentos**.
- Para quien quiere una **referencia rápida y clara** de los 50 conceptos que de verdad importan.

No necesitas instalar nada para empezar: puedes ejecutar todos los ejemplos en la **consola de tu navegador** (abre las herramientas de desarrollador con **F12** y ve a la pestaña *Console*).

Cómo usar este curso

1. Sigue las lecciones **en orden**. Cada una se apoya en la anterior.
2. **Escribe el código tú mismo**. No copies y pegues: teclear fija el aprendizaje.
3. Haz el **ejercicio** del final de cada lección antes de pasar a la siguiente.
4. Si algo no encaja, vuelve atrás. Repetir es normal y necesario.

Índice de las 50 lecciones

Módulo 1 — Fundamentos

1. ¿Qué es programar?
2. Variables
3. Tipos de datos
4. Operadores
5. Comentarios

Módulo 2 — Control de flujo

6. Condicionales: if
7. else y else if
8. Operadores lógicos
9. Comparaciones
10. switch
11. Operador ternario

Módulo 3 — Bucles

12. Bucle while
13. Bucle for
14. break y continue
15. for...of
16. Bucles anidados

Módulo 4 — Funciones

17. Declarar funciones
18. Parámetros y argumentos
19. return
20. Ámbito (scope)
21. Arrow functions
22. Funciones como valores
23. Recursión

Módulo 5 — Arrays

24. Crear arrays
25. Acceso por índice

- 26. [push, pop y compañía](#)
- 27. [map](#)
- 28. [filter](#)
- 29. [reduce](#)
- 30. [forEach](#)

Módulo 6 — Objetos

- 31. [Crear objetos](#)
- 32. [Propiedades](#)
- 33. [Métodos](#)
- 34. [this](#)
- 35. [Desestructuración](#)

Módulo 7 — Texto (strings)

- 36. [Métodos de string](#)
- 37. [Template literals](#)
- 38. [split y join](#)
- 39. [Buscar en texto](#)

Módulo 8 — Conceptos clave

- 40. [Mutabilidad](#)
- 41. [null y undefined](#)
- 42. [Coerción de tipos](#)
- 43. [Truthy y falsy](#)
- 44. [Errores y try/catch](#)

Módulo 9 — Asincronía

- 45. [Callbacks](#)
- 46. [Promesas](#)
- 47. [async / await](#)

Módulo 10 — Buenas prácticas

- 48. [Nombrar bien las variables](#)

- 49. [Depurar \(debugging\)](#)
 - 50. [Pensamiento computacional](#)
-

Recursos extra

- [Glosario de términos](#)
- [Chuleta de sintaxis JavaScript](#)
- [Qué hacer después de este curso](#)

Lección 1 — ¿Qué es programar?

Programar es **darle instrucciones a un ordenador para que haga una tarea**. El ordenador no piensa: hace exactamente lo que le dices, ni más ni menos. Si el resultado no es el esperado, casi siempre es porque las instrucciones no decían lo que creías.

Un **programa** es una lista de instrucciones escritas en un lenguaje que el ordenador entiende. Nosotros usaremos **JavaScript**, el lenguaje que viene incorporado en todos los navegadores web.

La clave del oficio no es memorizar comandos, sino aprender a **partir un problema grande en pasos pequeños y ordenados**. A eso se le llama *algoritmo*: una receta paso a paso.

Piensa en cómo le explicarías a alguien cómo hacer un café:

1. Llenar la cafetera con agua
2. Poner el café en el filtro
3. Encender la cafetera
4. Esperar a que termine
5. Servir en una taza

Eso ya es pensar como programador. El código solo es esa receta escrita de forma que la máquina la pueda seguir.

Tu primer programa

```
console.log("¡Hola, mundo!");
```

`console.log()` es una instrucción que **muestra un mensaje** en la consola. Lo que pongas entre comillas se imprime tal cual. Es la forma más común de "ver" lo que hace tu código mientras aprendes.

Cómo ejecutarlo

1. Abre tu navegador (Chrome, Firefox, Edge...).

2. Pulsa `F12` para abrir las herramientas de desarrollador.
3. Ve a la pestaña **Console**.
4. Escribe el código y pulsa `Enter`.

Verás aparecer `iHola, mundo!`. Acabas de ejecutar tu primer programa.

Ejercicio

Haz que la consola muestre tu nombre y, en una segunda línea, tu ciudad. Pista: necesitas **dos** instrucciones `console.log`.

Lección 2 — Variables

Una **variable** es una caja con nombre donde guardas un dato para usarlo más tarde. En vez de repetir un valor por todo el código, lo guardas una vez y lo llamas por su nombre.

En JavaScript se crean con `let` o `const`:

```
let edad = 25;
const nombre = "Ana";
```

- `let` → variable que **puede cambiar** después.
- `const` → variable **constante**, su valor no se puede reasignar.

```
let puntos = 10;
puntos = 20;           // ✅ permitido, usamos let

const pi = 3.14;
pi = 3;                // ❌ error: no se puede reasignar una const
```

Regla práctica: usa `const` por defecto. Cambia a `let` solo cuando sepas que el valor va a variar. Esto evita cambios accidentales y hace el código más fácil de entender.

Reglas de los nombres

- No pueden empezar por número (`2gatos` ❌, `gatos2` ✅).
- No llevan espacios (`mi edad` ❌, `miEdad` ✅).
- Distinguen mayúsculas: `edad` y `Edad` son **dos variables distintas**.

Usar una variable

```
const nombre = "Ana";
console.log("Hola, " + nombre); // Hola, Ana
```

El `+` aquí une (concatena) el texto con el valor de la variable.

Ejercicio

Crea una variable `precio` con valor `9.99` y otra `producto` con valor `"Curso"`. Luego muestra en consola: `El Curso cuesta 9.99 euros`.

Lección 3 — Tipos de datos

No todos los datos son iguales. Un número se trata distinto que un texto. Esos "tipos de cosa" se llaman **tipos de datos**. Conocer los básicos te evita muchos errores.

Los tipos esenciales

```
const numero = 42;           // number → números (enteros o decimales)
const texto = "Hola";        // string → texto, siempre entre comillas
const activo = true;          // boolean → solo true o false
const nada = null;            // null → "valor vacío" intencionado
let sinValor;                 // undefined → variable sin asignar todavía
```

- **number**: `7`, `-3`, `9.99`. No lleva comillas.
- **string**: `"Hola"`, `'Ana'`. Siempre entre comillas (dobles o simples).
- **boolean**: solo dos valores, `true` (verdadero) o `false` (falso). Base de toda decisión en programación.
- **null** y **undefined**: representan "ausencia de valor". Los verás a fondo en la Lección 41.

Cómo saber el tipo de algo

El operador `typeof` te dice el tipo de un valor:

```
console.log(typeof 42);       // "number"
console.log(typeof "Hola");   // "string"
console.log(typeof true);     // "boolean"
```

Cuidado: el número con comillas NO es número

```
const a = 5;           // number
const b = "5";         // string (texto que parece número)

console.log(a + a);    // 10 → suma
console.log(b + b);    // "55" → une dos textos
```

Este es uno de los tropiezos más comunes al empezar. Si un dato viene de un formulario o de internet, normalmente llega como **string**, aunque parezca número.

Ejercicio

Crea una variable de cada tipo (number, string, boolean) y usa `typeof` para comprobar cada una en la consola.

Lección 4 — Operadores

Los **operadores** son símbolos que hacen operaciones con los datos: sumar, comparar, unir textos... Son las herramientas básicas de cálculo.

Operadores aritméticos

```
console.log(5 + 3); // 8 suma
console.log(5 - 3); // 2 resta
console.log(5 * 3); // 15 multiplicación
console.log(6 / 3); // 2 división
console.log(7 % 3); // 1 resto (módulo): lo que sobra al dividir
```

El **módulo** (%) es muy útil: `7 % 3` es `1` porque 7 dividido entre 3 da 2 y sobra 1. Sirve, por ejemplo, para saber si un número es par: `n % 2 === 0`.

Asignación

```
let total = 10;
total = total + 5; // total ahora vale 15
total += 5; // forma corta de lo mismo → 20
total -= 3; // 17
total *= 2; // 34
```

`+=`, `-=`, `*=` son atajos: `x += 5` significa `x = x + 5`.

Concatenar texto

El `+` con strings **une** en vez de sumar:

```
const nombre = "Ana";
const saludo = "Hola, " + nombre + "!";
console.log(saludo); // Hola, Ana!
```

Incremento y decremento

```
let contador = 0;  
contador++; // suma 1 → 1  
contador--; // resta 1 → 0
```

Verás `++` constantemente en los bucles (Módulo 3).

Ejercicio

Tienes `const precio = 9.99` y `const cantidad = 3`. Calcula y muestra el total. Luego usa `%` para comprobar si `17` es par o impar.

Lección 5 — Comentarios

Un **comentario** es texto dentro del código que el ordenador **ignora**. Sirve para que tú (y otros) entendáis qué hace el código. No afecta a la ejecución.

Dos formas de comentar

```
// Esto es un comentario de una línea

/*
  Esto es un comentario
  de varias líneas
*/
```

- `//` → comenta hasta el final de la línea.
- `/* */` → comenta todo lo que haya en medio, aunque sean varias líneas.

Para qué sirven

1. **Explicar el "por qué", no el "qué".** El código ya dice qué hace; el comentario debe aclarar la intención o algo que no es obvio.

```
// ❌ obvio, sobra
let edad = 25; // asigna 25 a edad

// ✅ útil: explica una decisión
const IVA = 0.21; // 21% es el tipo general en España
```

2. **Desactivar código temporalmente** mientras pruebas:

```
console.log("línea A");
// console.log("línea B"); ← esta no se ejecuta
console.log("línea C");
```

Esto se llama "comentar" una línea y es una técnica diaria para depurar.

Buen consejo

No abuses. Demasiados comentarios obvios ensucian. El mejor código se explica casi solo con **buenos nombres de variables** (lo verás en la Lección 48). Comenta lo que sorprende, no lo evidente.

Ejercicio

Escribe tres `console.log`, comenta el del medio con `//` para que no se ejecute, y añade arriba un comentario de bloque explicando qué hace el programa.

Lección 6 — Condicionales: if

Hasta ahora el código corría de arriba a abajo sin desviarse. Con `if` el programa **toma decisiones**: ejecuta un bloque de código **solo si** una condición es verdadera.

```
const edad = 20;

if (edad >= 18) {
  console.log("Eres mayor de edad");
}
```

Estructura:

```
if (condición) {
  // código que se ejecuta SÓLO si la condición es true
}
```

- Entre paréntesis va la **condición**: algo que es `true` o `false`.
- Entre llaves `{ }` va el código que se ejecuta si se cumple.

La condición es un booleano

Recuerda la Lección 3: las decisiones se basan en `true / false`. La condición se evalúa a uno de esos dos valores.

```
const tieneEntrada = true;

if (tieneEntrada) {
  console.log("Puedes pasar");
}
```

Si `tieneEntrada` fuese `false`, el mensaje no aparecería.

Comparaciones típicas

```
if (saldo > 0)      { /* mayor que */ }  
if (intentos === 3) { /* igual a (ojo: tres iguales) */ }  
if (nota >= 5)     { /* mayor o igual */ }
```

Ejercicio

Crea una variable `temperatura`. Si es mayor que 30, muestra `"Hace calor"`. Prueba con varios valores cambiando la variable.

Lección 7 — else y else if

Con `if` solo cubres el caso "si se cumple". Con `else` añades el "si no se cumple", y con `else if` encadenas varias posibilidades.

else

```
const edad = 15;

if (edad >= 18) {
  console.log("Mayor de edad");
} else {
  console.log("Menor de edad");
}
```

Solo uno de los dos bloques se ejecuta, nunca ambos.

else if

Cuando hay más de dos caminos:

```
const nota = 7;

if (nota >= 9) {
  console.log("Sobresaliente");
} else if (nota >= 7) {
  console.log("Notable");
} else if (nota >= 5) {
  console.log("Aprobado");
} else {
  console.log("Suspenso");
}
```

Se evalúan **de arriba a abajo** y se ejecuta el **primero** que sea verdadero. El resto se ignora aunque también se cumplieran. Por eso el orden importa.

Cuidado con el orden

```
// ❌ MAL: el primero captura todo
if (nota >= 5) {
  console.log("Aprobado");
} else if (nota >= 9) {
  console.log("Sobresaliente"); // nunca se alcanza
}
```

Un 10 entra en `nota >= 5` y nunca llega al `>= 9`. Ordena siempre de **más específico/restrictivo a más general**.

Ejercicio

Pide la hora del día en una variable `hora` (0–23). Muestra `"Buenos días"` si es antes de las 12, `"Buenas tardes"` hasta las 20, y `"Buenas noches"` el resto.

Lección 8 — Operadores lógicos

A veces una decisión depende de **varias condiciones a la vez**. Los operadores lógicos las combinan.

Los tres operadores

Operador	Nombre	Es verdadero cuando...
&&	Y (AND)	ambas condiciones son verdaderas
	O (OR)	al menos una es verdadera
!	NO (NOT)	invierte: convierte true en false

&& (Y)

```
const edad = 25;
const tieneCarnet = true;

if (edad >= 18 && tieneCarnet) {
  console.log("Puede conducir");
}
```

Las **dos** cosas tienen que cumplirse.

|| (O)

```
const esFinde = false;
const esFestivo = true;

if (esFinde || esFestivo) {
  console.log("Hoy no se trabaja");
}
```

Basta con que **una** se cumpla.

! (NO)

```
const lloviendo = false;

if (!lloviendo) {
  console.log("Sal a pasear"); // se ejecuta: !false = true
}
```

! da la vuelta al valor. **!true** es **false** y **!false** es **true**.

Combinarlos

```
const edad = 20;
const invitado = false;

if (edad >= 18 && !invitado) {
  console.log("Entrada de pago");
}
```

Usa paréntesis cuando mezcles varios para dejar clara la intención: **(a && b) || c**.

Ejercicio

Una persona puede entrar a la piscina si **sabe nadar** Y (es mayor de edad O va acompañada).
Crea tres variables booleanas y escribe el **if**.

Lección 9 — Comparaciones

Comparar valores da siempre un booleano (`true` o `false`). Es la base de toda condición.

Operadores de comparación

```
console.log(5 > 3);    // true   mayor que
console.log(5 < 3);    // false  menor que
console.log(5 >= 5);   // true   mayor o igual
console.log(5 <= 4);   // false  menor o igual
console.log(5 === 5);  // true   igual (valor Y tipo)
console.log(5 !== 3);  // true   distinto
```

La trampa más famosa: `===` vs `==`

JavaScript tiene dos formas de comparar igualdad:

```
console.log(5 === "5"); // false → tipos distintos (number vs string)
console.log(5 == "5");  // true  → iconvierte y compara solo el valor!
```

- `===` compara **valor y tipo**. Es estricto y predecible.
- `==` convierte tipos por detrás y produce resultados confusos.

Regla de oro: usa siempre `===` y `!==`. Olvídate de `==`. Te ahorrará horas de errores difíciles de encontrar.

```
const entrada = "5";           // viene de un formulario, es string

if (entrada === 5) { ... }      // false → ojo, no es number
if (Number(entrada) === 5) { ... } // ✅ convierte primero
```

Comparar textos

Funciona alfabéticamente y distingue mayúsculas:

```
console.log("a" === "a");    // true
console.log("a" === "A");    // false → distinta mayúscula
```

Ejercicio

Comprueba qué devuelven `"10" === 10`, `"10" == 10` y `Number("10") === 10`. Razona por qué cada uno da lo que da.

Lección 10 — switch

Cuando comparas **una misma variable** contra muchos valores posibles, una cadena larga de `else if` se vuelve pesada. `switch` lo hace más legible.

De else if a switch

```
// Con else if
const dia = "lunes";
if (dia === "lunes") {
  console.log("Empieza la semana");
} else if (dia === "viernes") {
  console.log("¡Casi finde!");
} else {
  console.log("Día normal");
}
```

Lo mismo con `switch`:

```
switch (dia) {
  case "lunes":
    console.log("Empieza la semana");
    break;
  case "viernes":
    console.log("¡Casi finde!");
    break;
  default:
    console.log("Día normal");
}
```

Las piezas

- `switch (variable)` → el valor que se compara.
- `case valor:` → un caso posible. Compara con `===`.
- `break;` → **corta** el switch. Sin él, sigue ejecutando los casos siguientes (un error muy común).

- `default:` → lo que pasa si ningún caso coincide (como el `else`).

El peligro de olvidar break

```
switch (fruta) {  
  case "manzana":  
    console.log("Roja");  
    // sin break → itambién ejecuta el siguiente!  
  case "plátano":  
    console.log("Amarillo");  
    break;  
}
```

Si `fruta` es `"manzana"`, imprime **Roja Y Amarillo**. Pon siempre `break` salvo que quieras ese efecto a propósito.

Ejercicio

Haz un `switch` sobre una variable `nota` con valores `"A"`, `"B"`, `"C"` que imprima `"Excelente"`, `"Bien"`, `"Mejorable"`, y un `default` para cualquier otra cosa.

Lección 11 — Operador ternario

El **operador ternario** es un `if/else` escrito en una sola línea. Sirve para elegir entre dos valores de forma compacta.

Sintaxis

```
condición ? valorSiVerdadero : valorSiFalso
```

Se lee: "¿Se cumple la condición? Entonces lo de la izquierda; si no, lo de la derecha."

```
const edad = 20;
const mensaje = edad >= 18 ? "Mayor" : "Menor";
console.log(mensaje); // Mayor
```

Equivale a:

```
let mensaje;
if (edad >= 18) {
  mensaje = "Mayor";
} else {
  mensaje = "Menor";
}
```

Cuatro líneas convertidas en una. Por eso es muy usado para **asignar un valor según una condición**.

Casos típicos

```
const precio = 100;
const descuento = precio > 50 ? 0.1 : 0;

const usuario = null;
const nombre = usuario ? usuario : "Invitado";
```

Cuándo NO usarlo

El ternario brilla en casos cortos. Si la lógica es compleja o anidas varios, vuelve a `if/else`: la legibilidad manda.

```
// ❌ difícil de leer
const x = a ? (b ? 1 : 2) : (c ? 3 : 4);

// ✅ mejor un if/else normal
```

Ejercicio

Usa un ternario para guardar en `acceso` el texto `"Permitido"` si `edad >= 18`, o `"Denegado"` si no. Muestra el resultado.

Lección 12 — Bucle while

Un **bucle** repite un bloque de código varias veces sin que lo escribas mil veces. `while` repite **mientras** una condición sea verdadera.

Sintaxis

```
while (condición) {  
  // se repite mientras la condición sea true  
}
```

```
let contador = 1;  
  
while (contador <= 3) {  
  console.log("Vuelta " + contador);  
}  
// Vuelta 1  
// Vuelta 2  
// Vuelta 3
```

Las tres partes de todo bucle

1. **Inicio:** `let contador = 1;` (antes del bucle).
2. **Condición:** `contador <= 3` (cuándo seguir).
3. **Avance:** `contador++` (acercarse al final).

Si falta el avance, la condición nunca deja de cumplirse y el bucle no para jamás.

```
let n = 1;  
while (n <= 3) {  
  console.log(n);  
  // ifalta n++! → se repite para siempre y cuelga el navegador  
}
```

Es el error número uno con `while`. Asegúrate **siempre** de que algo dentro del bucle acerca la condición a ser falsa. Si lo provocas, cierra la pestaña para detenerlo.

Cuándo usar while

Cuando **no sabes de antemano cuántas veces** repetirás. Ejemplo: "sigue pidiendo la contraseña mientras sea incorrecta". Si sabes el número exacto de vueltas, el `for` (siguiente lección) es más cómodo.

Ejercicio

Usa un `while` para imprimir los números del 10 al 1 en orden descendente.

Lección 13 — Bucle for

El `for` es el bucle más usado. Junta las tres partes (inicio, condición, avance) en una sola línea, perfecto cuando **sabes cuántas veces** quieres repetir.

Sintaxis

```
for (inicio; condición; avance) {  
  // código que se repite  
}
```

```
for (let i = 1; i <= 5; i++) {  
  console.log("Número " + i);  
}  
// Número 1 ... Número 5
```

Compáralo con el `while` de la lección anterior: las mismas tres partes, pero todas juntas y a la vista.

Desglose

- `let i = 1` → se ejecuta **una vez** al empezar.
- `i <= 5` → se comprueba **antes de cada vuelta**; si es false, termina.
- `i++` → se ejecuta **al final de cada vuelta**.

La variable `i` (de *index*, índice) es el contador. Por convención se llama `i`, luego `j`, `k`.

Recorrer empezando en 0

En programación se cuenta desde **0** muchísimo (lo verás con los arrays). Patrón clásico:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);    // 0, 1, 2, 3, 4  
}
```

Fíjate: `i < 5` con inicio en 0 da **5 vueltas** (0,1,2,3,4). Este patrón aparece por todas partes.

Contar de dos en dos o hacia atrás

```
for (let i = 0; i <= 10; i += 2) { ... } // pares: 0,2,4...  
for (let i = 5; i > 0; i--) { ... }      // 5,4,3,2,1
```

Ejercicio

Con un `for`, imprime la tabla de multiplicar del 7 (del 7×1 al 7×10).

Lección 14 — break y continue

Dos palabras que controlan el flujo **dentro** de un bucle: `break` lo corta del todo, `continue` salta a la siguiente vuelta.

break: salir del bucle

```
for (let i = 1; i <= 10; i++) {  
  if (i === 5) {  
    break;          // sale del bucle por completo  
  }  
  console.log(i);   // 1, 2, 3, 4  
}
```

Cuando `i` llega a 5, `break` detiene el bucle. Útil para **dejar de buscar en cuanto encuentras** lo que querías.

```
const numeros = [3, 8, 1, 9, 4];  
for (let i = 0; i < numeros.length; i++) {  
  if (numeros[i] > 5) {  
    console.log("Primer número >5: " + numeros[i]);  
    break;    // no hace falta seguir mirando  
  }  
}
```

continue: saltar a la siguiente vuelta

```
for (let i = 1; i <= 5; i++) {  
  if (i % 2 === 0) {  
    continue;      // salta los pares  
  }  
  console.log(i);   // 1, 3, 5  
}
```

`continue` no termina el bucle: solo **se salta el resto de esa vuelta** y pasa a la siguiente.

Diferencia clave

- `break` → "ya he terminado, salgo del bucle".
- `continue` → "esta vuelta no me interesa, pasa a la siguiente".

Ejercicio

Recorre los números del 1 al 20. Salta con `continue` los múltiplos de 3, y usa `break` para parar en cuanto pases de 15.

Lección 15 — for...of

`for...of` recorre directamente **cada elemento** de una lista (array) o texto, sin tener que manejar índices. Es más limpio cuando solo quieres los valores.

Comparación

```
const frutas = ["manzana", "pera", "uva"];

// for clásico: con índice
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}

// for...of: directo al valor
for (const fruta of frutas) {
  console.log(fruta);
}
```

Ambos imprimen lo mismo, pero `for...of` se lee como una frase: "para cada fruta de frutas...". Sin `i`, sin `length`, sin `[i]`: menos sitios donde equivocarse.

Sintaxis

```
for (const elemento of coleccion) {
  // usa "elemento" en cada vuelta
}
```

Se usa `const` porque en cada vuelta `elemento` es un valor nuevo.

También recorre texto

```
for (const letra of "Hola") {  
  console.log(letra);  // H, o, l, a  
}
```

Cuándo usar cada uno

- **for...of** → cuando solo necesitas los valores. La opción por defecto para recorrer listas.
- **for clásico** → cuando necesitas el **índice** (la posición), o saltar de dos en dos, o ir hacia atrás.

Ejercicio

Tienes `const precios = [10, 25, 7, 40]`. Recórrelos con `for...of` y suma todos en una variable `total`. Muestra el total.

Lección 16 — Bucles anidados

Un **bucle anidado** es un bucle dentro de otro. El de dentro se ejecuta **completo** en cada vuelta del de fuera. Sirve para trabajar con tablas, cuadrículas y combinaciones.

Ejemplo

```
for (let i = 1; i <= 3; i++) {  
  for (let j = 1; j <= 2; j++) {  
    console.log("i=" + i + ", j=" + j);  
  }  
}
```

Salida:

```
i=1, j=1  
i=1, j=2  
i=2, j=1  
i=2, j=2  
i=3, j=1  
i=3, j=2
```

El bucle de fuera (**i**) da 3 vueltas; por **cada una**, el de dentro (**j**) da 2 vueltas completas. Total: $3 \times 2 = 6$ **ejecuciones**.

Para qué sirve

Cualquier cosa con dos dimensiones: filas y columnas, un tablero, todas las parejas posibles...

```
// Dibujar una cuadrícula de asteriscos 3x4
for (let fila = 0; fila < 3; fila++) {
  let linea = "";
  for (let col = 0; col < 4; col++) {
    linea += "*";
  }
  console.log(linea);    // **** (tres veces)
}
```

Cuidado con el coste

Anidar multiplica el trabajo. Dos bucles de 1.000 vueltas son **un millón** de ejecuciones. No es malo, pero tenlo en cuenta: si puedes resolver algo sin anidar, suele ser más eficiente.

Ejercicio

Usa dos bucles anidados para imprimir todas las tablas de multiplicar del 1 al 5 (el de fuera la tabla, el de dentro del 1 al 10).

Lección 17 — Declarar funciones

Una **función** es un bloque de código con nombre que puedes ejecutar cuando quieras, las veces que quieras. Escribes la receta una vez y la reutilizas. Es la herramienta más importante para no repetirte.

Declarar y llamar

```
function saludar() {  
  console.log("¡Hola!");  
}  
  
saludar();    // ¡Hola!  
saludar();    // ¡Hola!  (la llamamos otra vez)
```

Dos pasos distintos:

1. **Declarar**: defines qué hace la función (no se ejecuta todavía).
2. **Lllamar/invocar**: `saludar()` con los paréntesis. **Ahí** se ejecuta.

Por qué usarlas

Imagina que validas una edad en diez sitios del programa. Sin función, copias la lógica diez veces. Con función, la escribes una vez y la llamas. Si hay que corregir algo, lo cambias en **un solo lugar**.

```
function mostrarSeparador() {  
  console.log("-----");  
}  
  
console.log("Sección 1");  
mostrarSeparador();  
console.log("Sección 2");  
mostrarSeparador();
```

La regla DRY

Don't Repeat Yourself ("no te repitas"). Si te ves copiando el mismo código, probablemente debería ser una función. Es una de las señales de buen código.

Ejercicio

Crea una función `pedirCafe` que imprima los pasos para hacer un café (varios `console.log`). Llámala dos veces.

Lección 18 — Parámetros y argumentos

Una función fija siempre hace lo mismo. Con **parámetros**, le pasas datos para que trabaje con valores distintos cada vez. Eso la hace flexible.

Pasar datos a una función

```
function saludar(nombre) {  
  console.log("Hola, " + nombre);  
}  
  
saludar("Ana");    // Hola, Ana  
saludar("Luis");   // Hola, Luis
```

- **Parámetro:** `nombre`, la variable que recibe la función (al declararla).
- **Argumento:** `"Ana"`, el valor real que le pasas (al llamarla).

La misma función, resultados distintos según lo que le des.

Varios parámetros

Se separan por comas, **en orden**:

```
function sumar(a, b) {  
  console.log(a + b);  
}  
  
sumar(3, 5);    // 8  
sumar(10, 2);   // 12
```

El orden importa: el primer argumento va al primer parámetro, y así sucesivamente.

Valores por defecto

Puedes dar un valor por si no se pasa el argumento:


```
function saludar(nombre = "amigo") {  
  console.log("Hola, " + nombre);  
}
```

```
saludar();           // Hola, amigo  
saludar("Ana");      // Hola, Ana
```

Un error típico

```
function area(base, altura) {  
  console.log(base * altura);  
}
```

```
area(5); // base=5, altura=undefined → 5 * undefined = NaN
```

Si olvidas un argumento, el parámetro queda `undefined` y aparece `NaN` (*Not a Number*). Pasa siempre todos los que la función espera.

Ejercicio

Crea una función `precioConIVA(precio)` que muestre el precio multiplicado por 1.21. Pruébala con 100 y con 9.99.

Lección 19 — return

`return` hace que una función **devuelva un resultado** para usarlo después. Hasta ahora las funciones solo imprimían; con `return` te dan un valor que puedes guardar, sumar o pasar a otra función.

Imprimir vs devolver

```
// Solo imprime: el resultado se pierde
function sumarMal(a, b) {
  console.log(a + b);
}

// Devuelve: el resultado se puede reutilizar
function sumar(a, b) {
  return a + b;
}

const resultado = sumar(3, 5);
console.log(resultado);           // 8
console.log(sumar(2, 2) * 10); // 40 → operamos con lo devuelto
```

`console.log` solo **muestra** algo en pantalla. `return` **entrega** un valor al código que llamó a la función. Son cosas distintas, y esta confusión es muy habitual al empezar.

return corta la función

En cuanto se ejecuta un `return`, la función **termina**. Lo que haya debajo no se ejecuta:

```
function comprobar(edad) {
  if (edad < 18) {
    return "Menor"; // sale aquí
  }
  return "Mayor"; // solo se llega si edad >= 18
}
```

Sin return

Una función sin `return` devuelve `undefined`. Está bien si solo querías que hiciera algo (como imprimir), pero no podrás reutilizar un resultado.

Encadenar funciones

```
function doble(n) { return n * 2; }  
function masUno(n) { return n + 1; }  
  
console.log(masUno(doble(5))); // doble(5)=10 → masUno(10)=11
```

Ejercicio

Crea `function esPar(n)` que **devuelva** `true` o `false` según si `n` es par. Úsala dentro de un `if`.

Lección 20 — Ámbito (scope)

El **ámbito** (*scope*) define **dónde existe** una variable y desde dónde se puede usar. Entenderlo evita errores del tipo "esa variable no está definida".

Variables locales

Una variable creada **dentro** de una función solo existe ahí dentro:

```
function calcular() {  
  const local = 10;  
  console.log(local);  // ✅ funciona  
}  
  
calcular();  
console.log(local);    // ❌ Error: local is not defined
```

`local` "nace" al entrar en la función y "muere" al salir. Fuera no existe. Esto es bueno: cada función tiene su propio espacio y no se pisan entre ellas.

Variables globales

Una variable creada **fuera** de toda función se ve desde cualquier sitio:

```
const global = 100;  
  
function mostrar() {  
  console.log(global);  // ✅ las funciones ven las globales  
}
```

Las llaves crean ámbito

Con `let` y `const`, **cada bloque** `{ }` tiene su ámbito (un `if`, un `for` ...):

```
if (true) {  
  let x = 5;  
}  
console.log(x); // ✖ x solo existía dentro del if
```

Por qué importa

Mantener las variables en el ámbito **más pequeño posible** hace el código más seguro y fácil de seguir. Si una variable solo se usa dentro de una función, declárala ahí, no fuera. Las globales en exceso son fuente de errores difíciles.

Ejercicio

Declara una variable dentro de un `for`. Intenta usarla fuera del bucle y observa el error. Luego declárala antes del bucle para que sí funcione.

Lección 21 — Arrow functions

Las **arrow functions** (funciones flecha) son una forma más corta de escribir funciones. Verás esta sintaxis en todas partes en JavaScript moderno, sobre todo con `map`, `filter` y compañía (Módulo 5).

De función normal a flecha

```
// Función normal
function sumar(a, b) {
  return a + b;
}

// Arrow function equivalente
const sumar = (a, b) => {
  return a + b;
};
```

Cambios: desaparece la palabra `function`, el nombre va delante con `const`, y aparece la flecha `=>` entre los paréntesis y las llaves.

Versión aún más corta

Si la función **solo devuelve un valor**, puedes quitar las llaves y el `return`:

```
const sumar = (a, b) => a + b;
const doble = n => n * 2;           // un solo parámetro: paréntesis opcionales
const saludar = () => "Hola";       // sin parámetros: paréntesis vacíos
```

Esto se llama *return implícito*: lo que pones después de `=>` se devuelve automáticamente.

Cuándo las usarás

Sobre todo como **funciones cortas que pasas a otras funciones**:

```
const numeros = [1, 2, 3];  
const dobles = numeros.map(n => n * 2); // [2, 4, 6]
```

No te preocupes si `map` aún no te suena: lo verás pronto. Solo familiarízate con la sintaxis `n => n * 2`.

¿Cuál usar?

Para funciones normales con nombre, ambas valen. Las flecha brillan en código corto y "de paso". Hay una diferencia técnica con `this` (Lección 34) que de momento puedes ignorar.

Ejercicio

Reescribe como arrow function: `function cuadrado(n) { return n * n; }`. Hazlo primero con llaves y luego en versión corta de una línea.

Lección 22 — Funciones como valores

En JavaScript, una función es **un valor más**, como un número o un texto. Puedes guardarla en una variable y **pasarla a otra función**. Esta idea desbloquea muchísimo del lenguaje.

Una función guardada en variable

Ya lo viste con las arrow functions:

```
const saludar = () => "Hola";
console.log(saludar()); // Hola
```

`saludar` es una variable que contiene una función.

Pasar una función a otra función

Aquí está lo potente: puedes dar una función **como argumento**.

```
function ejecutarDosVeces(accion) {
  accion();
  accion();
}

ejecutarDosVeces(() => console.log("¡Hola!"));
// ¡Hola!
// ¡Hola!
```

`ejecutarDosVeces` recibe una función (`accion`) y la llama dos veces. A una función que recibe otra función se le llama *función de orden superior*.

Callback

La función que pasas como argumento se llama **callback** ("función de retorno"). El término aparecerá mucho. La idea: "toma esta función y llámala tú cuando toque".


```
function procesar(numero, transformacion) {  
  return transformacion(numero);  
}  
  
console.log(procesar(5, n => n * 2));    // 10  
console.log(procesar(5, n => n + 100)); // 105
```

La misma función `procesar` hace cosas distintas según el callback que le pases. Flexibilidad total.

Por qué importa

Métodos como `map`, `filter`, `forEach` (Módulo 5) y casi todo el código asíncrono (Módulo 9) funcionan así: les pasas una función que ellos ejecutan por ti. Entender esto ahora te facilitará todo lo que viene.

Ejercicio

Crea una función `aplicar(valor, fn)` que devuelva `fn(valor)`. Pásale un número y una arrow function que lo eleve al cuadrado.

Lección 23 — Recursión

La **recursión** ocurre cuando una función **se llama a sí misma**. Suena raro, pero es una forma elegante de resolver problemas que se repiten en versiones más pequeñas.

La idea

Una función recursiva necesita **dos cosas** sí o sí:

1. **Caso base**: cuándo parar (sin él, se llama para siempre).
2. **Caso recursivo**: se llama a sí misma con un problema **más pequeño**.

Ejemplo: cuenta atrás

```
function cuentaAtras(n) {  
  if (n === 0) {          // caso base: paramos  
    console.log("¡Despegue!");  
    return;  
  }  
  console.log(n);  
  cuentaAtras(n - 1);    // caso recursivo: con n más pequeño  
}  
  
cuentaAtras(3);  
// 3  
// 2  
// 1  
// ¡Despegue!
```

Cada llamada reduce `n` en 1, acercándose al caso base `n === 0`. Ahí para.

Ejemplo clásico: factorial

El factorial de 4 es $4 \times 3 \times 2 \times 1$. Se define en términos de sí mismo: `factorial(n) = n × factorial(n-1)`.

```
function factorial(n) {  
  if (n <= 1) return 1;           // caso base  
  return n * factorial(n - 1);    // caso recursivo  
}
```

```
console.log(factorial(4)); // 24
```

```
function infinita(n) {  
  return infinita(n - 1); // nunca para → error de pila  
}
```

Esto produce *"Maximum call stack size exceeded"*: la función se llamó tantas veces que se quedó sin memoria. Siempre define un caso base alcanzable.

¿Recursión o bucle?

Casi todo lo recursivo se puede hacer con un bucle, y al revés. La recursión brilla en estructuras "en árbol" (carpetas dentro de carpetas, menús anidados). De momento basta con entender la idea: una función que se llama a sí misma con un problema menor.

Ejercicio

Escribe una función recursiva `sumar(n)` que sume todos los números de 1 a `n`. Pista: `sumar(n) = n + sumar(n-1)`, y el caso base es `sumar(0) = 0`.

Lección 24 — Crear arrays

Un **array** (lista) es una colección ordenada de valores guardados en una sola variable. En vez de `fruta1`, `fruta2`, `fruta3`, tienes **una** lista con todas.

Crear un array

```
const frutas = ["manzana", "pera", "uva"];
const numeros = [10, 20, 30, 40];
const mezcla = ["Ana", 25, true];    // puede mezclar tipos
const vacio = [];                    // array vacío para llenar luego
```

Se escriben entre corchetes `[ ]`, con los elementos separados por comas.

Cuántos elementos tiene

La propiedad `.length` te da el tamaño:

```
const frutas = ["manzana", "pera", "uva"];
console.log(frutas.length);    // 3
```

Recorrer un array

Aquí se juntan dos cosas que ya sabes: arrays y bucles.

```
const frutas = ["manzana", "pera", "uva"];

for (const fruta of frutas) {
  console.log(fruta);
}
```

Esta combinación —guardar muchos datos en una lista y recorrerlos— es de lo más útil que harás como programador.

¿const y aun así puedo cambiarlo?

Sí. `const` impide **reasignar** la variable, pero el contenido del array **sí** se puede modificar (añadir, quitar). Lo verás en la Lección 26 y se explica del todo en Mutabilidad (Lección 40).

```
const lista = [1, 2];  
lista.push(3);    // ✅ permitido: modificamos el contenido  
lista = [9, 9];   // ❌ error: reasignar la variable
```

Ejercicio

Crea un array con los nombres de tres amigos. Muestra cuántos hay con `.length` y luego recórrelos imprimiendo cada uno.

Lección 25 — Acceso por índice

Cada elemento de un array tiene una posición numérica llamada **índice**. Con él accedes a un elemento concreto. La clave que sorprende a todo principiante: **se empieza a contar desde 0**.

El índice empieza en 0

```
const frutas = ["manzana", "pera", "uva"];  
//           0       1       2       ← índices  
  
console.log(frutas[0]); // manzana (el PRIMERO)  
console.log(frutas[1]); // pera  
console.log(frutas[2]); // uva      (el ÚLTIMO)
```

El primer elemento es el `[0]`, no el `[1]`. Es la causa de muchos errores al empezar, así que grábatelo.

El último elemento

Como el último índice es `length - 1`:

```
const frutas = ["manzana", "pera", "uva"];  
console.log(frutas[frutas.length - 1]); // uva
```

JavaScript moderno también permite `.at(-1)` para el último:

```
console.log(frutas.at(-1)); // uva
```

Índice que no existe

```
const frutas = ["manzana", "pera"];  
console.log(frutas[5]); // undefined → no hay nada ahí
```

No da error: devuelve `undefined` . Cuidado, porque luego puede romper otra parte del código.

Cambiar un elemento

```
const frutas = ["manzana", "pera", "uva"];
frutas[1] = "plátano";
console.log(frutas); // ["manzana", "plátano", "uva"]
```

Ejercicio

Con `const dias = ["lunes", "martes", "miércoles"]`, imprime el primero y el último usando índices. Luego cambia `"martes"` por `"sábado"` .

Lección 26 — push, pop y compañía

Los arrays traen **métodos**: funciones listas para añadir, quitar y manipular elementos. Estos cuatro son los del día a día.

Añadir y quitar al final

```
const pila = [1, 2, 3];

pila.push(4);    // añade al final → [1, 2, 3, 4]
pila.pop();      // quita el último → [1, 2, 3] (y lo devuelve)
```

- `push(x)` → mete `x` al final.
- `pop()` → saca el último y te lo devuelve.

Añadir y quitar al principio

```
const cola = [1, 2, 3];

cola.unshift(0); // añade al principio → [0, 1, 2, 3]
cola.shift();    // quita el primero → [1, 2, 3] (y lo devuelve)
```

Otros métodos muy útiles

```
const frutas = ["manzana", "pera", "uva"];

console.log(frutas.includes("pera")); // true → ¿está?
console.log(frutas.indexOf("uva"));   // 2    → ¿en qué posición?
console.log(frutas.join(", "));       // "manzana, pera, uva" → a texto
```

- `includes(x)` → `true` / `false` según si `x` está en el array.
- `indexOf(x)` → el índice de `x`, o `-1` si no está.
- `join(separador)` → une todo en un string.

push devuelve el nuevo tamaño

Detalle útil: `push` devuelve la nueva longitud, no el array.

```
const a = [1, 2];  
console.log(a.push(3)); // 3 (la nueva length), y a es [1, 2, 3]
```

Ejercicio

Parte de `const cesta = ["pan", "leche"]`. Añade `"huevos"` al final, `"agua"` al principio, comprueba con `includes` si hay `"leche"`, y muestra la cesta como texto con `join`.

Lección 27 — map

`map` crea un **array nuevo** transformando cada elemento del original. Es la forma moderna y limpia de "aplicar algo a toda la lista". Aquí ves para qué servían los callbacks de la Lección 22.

La idea

```
const numeros = [1, 2, 3, 4];
const dobles = numeros.map(n => n * 2);

console.log(dobles);    // [2, 4, 6, 8]
console.log(numeros);   // [1, 2, 3, 4] → iel original NO cambia!
```

`map` recorre cada elemento, le aplica la función que le pasas, y mete el resultado en un array nuevo. El original queda intacto.

Cómo leerlo

```
numeros.map(n => n * 2)
//           └─ por cada "n" del array, devuelve "n * 2"
```

La función recibe un elemento (`n`) y **devuelve** lo que quieres que aparezca en su lugar. Imprescindible que devuelva algo (con `return` o return implícito).

Ejemplos

```
const nombres = ["ana", "luis"];
const mayus = nombres.map(nombre => nombre.toUpperCase());
console.log(mayus);    // ["ANA", "LUIS"]

const precios = [10, 20, 30];
const conIVA = precios.map(p => p * 1.21);
console.log(conIVA);   // [12.1, 24.2, 36.3]
```

map vs for

Esto:

```
const dobles = [];  
for (const n of numeros) {  
  dobles.push(n * 2);  
}
```

...es lo mismo que `numeros.map(n => n * 2)`, pero `map` lo dice en una línea y deja claro que estás **transformando una lista en otra**. Por eso se prefiere.

Regla clave

`map` siempre devuelve un array **del mismo tamaño** que el original. Si quieres quitar elementos, ese es trabajo de `filter` (siguiente lección).

Ejercicio

Dado `const temperaturas = [0, 20, 30]` en Celsius, usa `map` para convertirlas a Fahrenheit (`C * 9/5 + 32`).

Lección 28 — filter

`filter` crea un **array nuevo** con solo los elementos que **cumplen una condición**. Donde `map` transforma, `filter` selecciona.

La idea

```
const numeros = [1, 2, 3, 4, 5, 6];
const pares = numeros.filter(n => n % 2 === 0);

console.log(pares);    // [2, 4, 6]
console.log(numeros);  // [1, 2, 3, 4, 5, 6] → original intacto
```

La función que pasas debe devolver `true` o `false`. Si devuelve `true`, el elemento **pasa el filtro** y entra en el array nuevo; si devuelve `false`, se descarta.

Cómo leerlo

```
numeros.filter(n => n % 2 === 0)
//           ↳ "¿este n cumple? quédatelo si la condición es true"
```

Ejemplos

```
const edades = [15, 22, 17, 30];
const mayores = edades.filter(e => e >= 18);
console.log(mayores);    // [22, 30]

const palabras = ["sol", "elefante", "luz", "ordenador"];
const largas = palabras.filter(p => p.length > 3);
console.log(largas);     // ["elefante", "ordenador"]
```

map vs filter

- `map` → mismo número de elementos, transformados.
- `filter` → menos (o igual) elementos, sin transformar.

Encadenarlos

Lo bonito es que devuelven arrays, así que puedes **encadenar**:

```
const numeros = [1, 2, 3, 4, 5, 6];
const resultado = numeros
  .filter(n => n % 2 === 0)    // [2, 4, 6]
  .map(n => n * 10);           // [20, 40, 60]

console.log(resultado);        // [20, 40, 60]
```

Primero filtras los pares, luego los multiplicas. Léible de arriba a abajo.

Ejercicio

Dado `const productos = [12, 50, 8, 99, 30]` (precios), filtra los que cuestan más de 20 € y luego, encadenando, súmales un 10 % con `map`.

Lección 29 — reduce

`reduce` combina **todos** los elementos de un array en **un único valor**: una suma, un total, un máximo... Es el más potente y el que más cuesta al principio, así que vamos despacio.

El caso más común: sumar

```
const numeros = [10, 20, 30];
const total = numeros.reduce((acumulador, n) => acumulador + n, 0);
console.log(total); // 60
```

Las piezas

```
array.reduce((acumulador, elemento) => ..., valorInicial)
```

- **acumulador**: el resultado que se va arrastrando vuelta a vuelta.
- **elemento**: el valor actual del array.
- **valorInicial**: con qué empieza el acumulador (aquí `0`).

Paso a paso con `[10, 20, 30]` y inicial `0`:

Vuelta	acumulador	elemento	resultado
1	0	10	10
2	10	20	30
3	30	30	60

El último resultado (`60`) es lo que devuelve `reduce` .

Otros usos

```
// Máximo
const max = [3, 9, 1, 7].reduce((a, b) => a > b ? a : b);
console.log(max);    // 9

// Contar / acumular texto
const carrito = [{ precio: 10 }, { precio: 25 }];
const totalCarrito = carrito.reduce((suma, item) => suma + item.precio, 0);
console.log(totalCarrito);    // 35
```

(Los objetos `{ precio: 10 }` los verás en el Módulo 6.)

No te agobies

`reduce` es el más abstracto del trío map/filter/reduce. Si al principio solo lo usas para **sumar totales**, ya le sacas el 80 % del provecho. La intuición llega con la práctica.

Ejercicio

Usa `reduce` para sumar `const gastos = [4.5, 12, 8, 20]`. Pon `0` como valor inicial.

Lección 30 — forEach

`forEach` ejecuta una función **por cada elemento** del array. A diferencia de `map`, **no devuelve nada nuevo**: lo usas cuando solo quieres *hacer algo* con cada elemento (imprimir, guardar...), no transformar la lista.

La idea

```
const frutas = ["manzana", "pera", "uva"];

frutas.forEach(fruta => {
  console.log("Me gusta la " + fruta);
});
// Me gusta la manzana
// Me gusta la pera
// Me gusta la uva
```

También te da el índice

La función recibe un segundo parámetro opcional: la posición.

```
frutas.forEach((fruta, i) => {
  console.log(i + ": " + fruta);
});
// 0: manzana
// 1: pera
// 2: uva
```

forEach vs map: la diferencia clave

```
const numeros = [1, 2, 3];

const r1 = numeros.map(n => n * 2);    // [2, 4, 6] ← devuelve array
const r2 = numeros.forEach(n => n * 2); // undefined ← NO devuelve nada
```


- `map` → cuando quieres **un array nuevo** transformado.
- `forEach` → cuando solo quieres **ejecutar una acción** por elemento y no necesitas resultado.

Un error común es usar `forEach` esperando un array de vuelta. Si necesitas el resultado, usa `map`.

forEach vs for...of

Hacen casi lo mismo. Dos diferencias prácticas:

- En `for...of` puedes usar `break` / `continue`; en `forEach` **no**.
- `forEach` es muy legible para acciones simples.

Elige el que te resulte más claro en cada caso.

Ejercicio

Tienes `const alumnos = ["Ana", "Luis", "Eva"]`. Usa `forEach` para imprimir `"1. Ana"`, `"2. Luis"`, `"3. Eva"` (ojo: el número es el índice + 1).

Lección 31 — Crear objetos

Un **objeto** agrupa datos relacionados bajo un mismo nombre, usando **parejas clave-valor**. Si un array es una lista numerada, un objeto es una ficha con etiquetas.

El problema que resuelve

Para describir una persona con variables sueltas:

```
const nombre = "Ana";
const edad = 25;
const ciudad = "Madrid";
```

Tres variables dispersas. Un objeto las mantiene juntas:

```
const persona = {
  nombre: "Ana",
  edad: 25,
  ciudad: "Madrid"
};
```

Anatomía

```
const persona = {
  clave: valor,          // cada línea: clave (etiqueta) : valor
  edad: 25,
};
```

- Entre llaves `{ }`.
- Cada propiedad es `clave: valor`.
- Separadas por comas.
- La clave es el nombre; el valor, el dato (de cualquier tipo: número, texto, array, incluso otro objeto).

Objetos dentro de objetos y arrays

```
const usuario = {
  nombre: "Luis",
  hobbies: ["leer", "correr"],    // un array dentro
  direccion: {                  // un objeto dentro
    ciudad: "Sevilla",
    cp: "41001"
  }
};
```

Así se modela información del mundo real: un usuario tiene una dirección, que a su vez tiene ciudad y código postal.

Array vs objeto

- **Array** `[ ]` → muchos elementos del mismo tipo, en orden, accedidos por número.
- **Objeto** `{ }` → datos variados de **una cosa**, accedidos por nombre.

A menudo se combinan: un array de objetos (una lista de personas) es la estructura más común en aplicaciones reales.

Ejercicio

Crea un objeto `libro` con las propiedades `titulo`, `autor`, `anio` y `disponible` (booleano). Rellénalo con un libro que conozcas.

Lección 32 — Propiedades

Las **propiedades** son los datos guardados en un objeto. Ahora veremos cómo leerlas, cambiarlas, añadir las y borrarlas.

Leer una propiedad

Dos formas:

```
const persona = { nombre: "Ana", edad: 25 };

console.log(persona.nombre);    // Ana    → notación de punto
console.log(persona["edad"]);  // 25     → notación de corchetes
```

La de **punto** es la habitual. La de **corchetes** sirve cuando el nombre de la propiedad está en una variable:

```
const campo = "nombre";
console.log(persona[campo]);    // Ana
```

Cambiar y añadir

```
const persona = { nombre: "Ana" };

persona.nombre = "Eva";        // cambia la existente
persona.edad = 30;             // añade una nueva (no existía)

console.log(persona);          // { nombre: "Eva", edad: 30 }
```

Si la propiedad existe, la actualizas; si no, se crea. Misma sintaxis.

Borrar una propiedad

```
delete persona.edad;  
console.log(persona);    // { nombre: "Eva" }
```

Propiedad que no existe

```
console.log(persona.telefono);    // undefined
```

Como con los arrays, no da error: devuelve `undefined` . Para comprobar si existe:

```
if (persona.telefono) {  
  console.log("Tiene teléfono");  
}
```

Recorrer todas las propiedades

```
const persona = { nombre: "Ana", edad: 25 };  
  
for (const clave in persona) {  
  console.log(clave + ": " + persona[clave]);  
}  
// nombre: Ana  
// edad: 25
```

Ojo: con objetos se usa `for...in` (claves), no `for...of` .

Ejercicio

Parte del objeto `libro` de la lección anterior. Cambia `disponible` a `false` , añade una propiedad `paginas` , y borra `anio` .

Lección 33 — Métodos

Un **método** es una función guardada dentro de un objeto. Así un objeto no solo tiene datos (propiedades), sino también **acciones** que puede realizar.

Una función como propiedad

```
const persona = {  
  nombre: "Ana",  
  saludar: function() {  
    console.log("Hola, soy Ana");  
  }  
};  
  
persona.saludar(); // Hola, soy Ana
```

`saludar` es un método: una propiedad cuyo valor es una función. Se llama con punto y paréntesis: `persona.saludar()`.

Sintaxis corta

JavaScript permite escribirlo más limpio, sin `: function`:

```
const persona = {  
  nombre: "Ana",  
  saludar() {  
    console.log("Hola");  
  }  
};
```

Ya has usado métodos

`array.push()`, `texto.toUpperCase()`, `array.map()` ... todos son métodos: funciones que "pertenecen" a ese valor. Ahora entiendes de dónde salen.

```
const lista = [1, 2, 3];  
lista.push(4);           // push es un método del array  
"hola".toUpperCase();   // toUpperCase es un método del string
```

Métodos con parámetros

```
const calculadora = {  
  sumar(a, b) {  
    return a + b;  
  }  
};  
  
console.log(calculadora.sumar(3, 5)); // 8
```

Y para usar los datos del propio objeto...

Un método a menudo necesita acceder a las propiedades de su objeto (por ejemplo, que `saludar` diga el `nombre` real). Para eso existe `this`, la siguiente lección.

Ejercicio

Crea un objeto `circulo` con una propiedad `radio` y un método `area()` que devuelva el área ($3.14 * \text{radio} * \text{radio}$). De momento, usa el valor del radio directamente; en la próxima lección lo haremos con `this`.

Lección 34 — this

`this` es una palabra especial que, dentro de un método, se refiere **al propio objeto**. Permite que un método use los datos de su objeto sin nombrarlo a pelo.

El problema

```
const persona = {  
  nombre: "Ana",  
  saludar() {  
    console.log("Hola, soy " + nombre);    // ❌ nombre no existe aquí suelto  
  }  
};
```

`nombre` a secas no funciona dentro del método. Hay que decir "el nombre **de este objeto**".

La solución: this

```
const persona = {  
  nombre: "Ana",  
  saludar() {  
    console.log("Hola, soy " + this.nombre);    // ✅  
  }  
};  
  
persona.saludar();    // Hola, soy Ana
```

`this.nombre` significa "la propiedad `nombre` del objeto sobre el que se llamó el método".

La gran ventaja: reutilizar

El mismo método sirve para objetos distintos:


```
const ana = {
  nombre: "Ana",
  saludar() { console.log("Soy " + this.nombre); }
};
const luis = {
  nombre: "Luis",
  saludar() { console.log("Soy " + this.nombre); }
};

ana.saludar();    // Soy Ana
luis.saludar();   // Soy Luis
```

`this` apunta al objeto que está **a la izquierda del punto** al llamar.

Una sutileza: las arrow functions **no** tienen su propio `this`. Por eso, para métodos de objeto, usa la **forma normal** (`saludar() { }`), no la flecha. Es la principal diferencia práctica entre ambas (recuerda Lección 21).

```
const obj = {
  nombre: "Ana",
  saludar: () => console.log(this.nombre)    // ❌ this no es el objeto
};
```

No te obsesiones con el porqué profundo ahora: la regla práctica es "métodos de objeto → función normal".

Ejercicio

Reescribe el objeto `circulo` de la lección anterior para que `area()` use `this.radio` en vez del valor directo.

Lección 35 — Desestructuración

La **desestructuración** es una forma rápida de **sacar valores** de un objeto o array y meterlos en variables. Ahorra código y se ve por todas partes en proyectos reales.

Desestructurar objetos

Sin desestructuración:

```
const persona = { nombre: "Ana", edad: 25 };  
const nombre = persona.nombre;  
const edad = persona.edad;
```

Con desestructuración, en una línea:

```
const persona = { nombre: "Ana", edad: 25 };  
const { nombre, edad } = persona;  
  
console.log(nombre);    // Ana  
console.log(edad);      // 25
```

Las variables deben llamarse **igual que las propiedades**. JavaScript las empareja por nombre.

Desestructurar arrays

Aquí se emparejan por **posición**, no por nombre:

```
const colores = ["rojo", "verde", "azul"];  
const [primero, segundo] = colores;  
  
console.log(primero);   // rojo  
console.log(segundo);   // verde
```

Uso muy común: en parámetros de función

```
function mostrar({ nombre, edad }) {  
  console.log(nombre + " tiene " + edad + " años");  
}  
  
mostrar({ nombre: "Luis", edad: 30 }); // Luis tiene 30 años
```

En vez de recibir el objeto entero y escribir `persona.nombre`, sacas directamente lo que necesitas. Muy frecuente en frameworks modernos.

Valores por defecto

```
const { ciudad = "Desconocida" } = { nombre: "Ana" };  
console.log(ciudad); // Desconocida (no existía en el objeto)
```

Ejercicio

Tienes `const producto = { nombre: "Curso", precio: 9.99, stock: 100 }`. Desestructura `nombre` y `precio` en una sola línea y muéstralos.

Lección 36 — Métodos de string

Los **strings** (textos) traen métodos para transformarlos y consultarlos. Como con los arrays, son funciones que "cuelgan" del texto con un punto.

Longitud

```
const texto = "Hola";  
console.log(texto.length); // 4 (length es propiedad, sin paréntesis)
```

Mayúsculas y minúsculas

```
console.log("hola".toUpperCase()); // "HOLA"  
console.log("HOLA".toLowerCase()); // "hola"
```

Útil para comparar sin que importe la capitalización:

```
const entrada = "Sí";  
if (entrada.toLowerCase() === "sí") { ... }
```

Quitar espacios sobrantes

```
console.log("  hola  ".trim()); // "hola"
```

`trim` quita espacios al principio y al final. Imprescindible para limpiar lo que el usuario escribe en formularios.

Acceder a un carácter

```
const texto = "Hola";
console.log(texto[0]);      // "H"  (como los arrays, empieza en 0)
console.log(texto.charAt(1)); // "o"
```

Reemplazar

```
console.log("Hola mundo".replace("mundo", "Ana")); // "Hola Ana"
console.log("a-b-c".replaceAll("-", "/"));         // "a/b/c"
```

Trocear

```
console.log("Hola mundo".slice(0, 4)); // "Hola"  (del 0 al 4 sin incluirlo)
```

Los métodos **no cambian** el texto original: devuelven uno **nuevo**.

```
const t = "hola";
t.toUpperCase();
console.log(t);           // "hola" → ¡no cambió!
const mayus = t.toUpperCase(); // hay que guardarlo
console.log(mayus);       // "HOLA"
```

Olvidar guardar el resultado es un error muy común. (Más sobre inmutabilidad en la Lección 40.)

Ejercicio

Dado `const sucio = " Hola Mundo "`, límpialo con `trim`, pásalo a minúsculas y reemplaza `"mundo"` por tu nombre. Encadena los métodos.

Lección 37 — Template literals

Los **template literals** son una forma mucho más cómoda de construir textos con variables dentro. Adiós a concatenar con mil `+`.

El problema con +

```
const nombre = "Ana";
const edad = 25;
const frase = "Hola, " + nombre + ". Tienes " + edad + " años.";
```

Difícil de leer y fácil de olvidar un espacio o un `+`.

La solución: comillas invertidas

Se usan backticks ``` (tecla normalmente junto al 1, o `Alt+96`) y se insertan variables con `${ }`:

```
const nombre = "Ana";
const edad = 25;
const frase = `Hola, ${nombre}. Tienes ${edad} años.`;
console.log(frase); // Hola, Ana. Tienes 25 años.
```

Mucho más claro: ves la frase final tal cual, con los huecos marcados.

Dentro de \${ } va cualquier expresión

No solo variables: cualquier cálculo.

```
const precio = 10;
console.log(`Con IVA: ${precio * 1.21} €`); // Con IVA: 12.1 €
console.log(`El doble de 5 es ${5 * 2}`); // El doble de 5 es 10
console.log(`Mayor de edad: ${edad >= 18}`); // Mayor de edad: true
```

Texto en varias líneas

Otra ventaja: los backticks respetan los saltos de línea.

```
const carta = `Hola Ana,  
  
Gracias por tu compra.  
  
Un saludo.`;
```

Con comillas normales esto necesitaría `\n` por todas partes.

Recomendación

Usa template literals **siempre** que mezcles texto y variables. Es el estándar en JavaScript moderno y hace el código mucho más legible.

Ejercicio

Reescribe con template literals: `"El producto " + nombre + " cuesta " + precio + " euros"`. Crea las variables y muéstralo.

Lección 38 — split y join

`split` convierte un **texto en array**, y `join` convierte un **array en texto**. Son la puerta entre strings y arrays, y los usarás muchísimo para procesar datos.

split: de texto a array

```
const frase = "uno,dos,tres";
const partes = frase.split(",");
console.log(partes); // ["uno", "dos", "tres"]
```

Le pasas el **separador** y te devuelve un array cortando por ahí.

```
"Hola mundo".split(" "); // ["Hola", "mundo"] → separa por espacios
"abc".split(""); // ["a", "b", "c"] → separa letra a letra
```

join: de array a texto

Es la operación inversa (ya la viste en la Lección 26):

```
const palabras = ["Hola", "mundo"];
console.log(palabras.join(" ")); // "Hola mundo"
console.log(palabras.join("-")); // "Hola-mundo"
```

Le pasas el separador con el que pegar los elementos.

La combinación es potente

Texto → array → lo manipulas → texto de vuelta:


```
const frase = "hola mundo cruel";

const resultado = frase
  .split(" ") // ["hola", "mundo", "cruel"]
  .map(palabra => palabra.toUpperCase()) // ["HOLA", "MUNDO", "CRUEL"]
  .join(" "); // "HOLA MUNDO CRUEL"

console.log(resultado); // HOLA MUNDO CRUEL
```

Aquí se juntan cuatro lecciones: `split`, `map`, métodos de string y `join`. Esto es programar de verdad: encadenar herramientas simples.

Caso típico: contar palabras

```
const texto = "el gato y el perro";
console.log(texto.split(" ").length); // 5 palabras
```

Ejercicio

Dado `const nombres = "ana,luis,eva"`, conviértelo en array con `split`, ponlos en mayúsculas con `map`, y únelos de nuevo separados por `" | "` con `join`.

Lección 39 — Buscar en texto

Comprobar si un texto contiene algo, dónde empieza o cómo empieza/termina es una tarea diaria: validar emails, filtrar resultados, detectar palabras...

¿Contiene algo? → includes

```
const frase = "Hola mundo";
console.log(frase.includes("mundo")); // true
console.log(frase.includes("adiós")); // false
```

Devuelve `true` / `false`. Es el método de búsqueda que más usarás.

¿Empieza o termina con...?

```
const archivo = "documento.pdf";
console.log(archivo.startsWith("doc")); // true
console.log(archivo.endsWith(".pdf")); // true
```

Perfecto para comprobar extensiones, prefijos, etc.

¿En qué posición está? → indexOf

```
const frase = "Hola mundo";
console.log(frase.indexOf("mundo")); // 5 (empieza en el índice 5)
console.log(frase.indexOf("xyz")); // -1 (no está)
```

Devuelve el índice donde empieza, o `-1` si no aparece. Recuerda: `-1` significa "no encontrado".

Ejemplo práctico: buscador simple

```
const productos = ["Camiseta roja", "Pantalón azul", "Camiseta verde"];
const busqueda = "camiseta";

const resultados = productos.filter(p =>
  p.toLowerCase().includes(busqueda.toLowerCase())
);

console.log(resultados); // ["Camiseta roja", "Camiseta verde"]
```

Fíjate en el truco: pasar **ambos** textos a minúsculas con `toLowerCase` antes de comparar, para que la búsqueda no distinga mayúsculas. Aquí se unen `filter`, `includes` y métodos de string.

Ejercicio

Tienes `const correos = ["ana@gmail.com", "luis@hotmail.com", "eva@gmail.com"]`. Filtra los que terminan en `"@gmail.com"` usando `endsWith`.

Lección 40 — Mutabilidad

Mutar significa modificar algo después de crearlo. Entender qué se puede mutar y qué no aclara muchos comportamientos "raros" de JavaScript, sobre todo con `const`.

Dos grupos de tipos

- **Primitivos** (number, string, boolean): **inmutables**. No se pueden cambiar por dentro.
- **Objetos y arrays**: **mutables**. Su contenido sí se puede cambiar.

Los primitivos no cambian

```
let texto = "hola";
texto.toUpperCase();    // genera "HOLA" pero no toca el original
console.log(texto);     // "hola"

texto = "adiós";        // esto NO es mutar: es reasignar la variable
```

Por eso los métodos de string devuelven uno nuevo (Lección 36): el original es intocable.

Los arrays y objetos sí cambian

```
const lista = [1, 2, 3];
lista.push(4);          // mutamos el contenido
console.log(lista);     // [1, 2, 3, 4]
```

El misterio del const

Esto confunde a todos al principio:

```
const lista = [1, 2];  
lista.push(3);    // ✅ permitido  
lista = [9, 9];   // ❌ error
```

`const` solo impide **reasignar** la variable (apuntarla a otra cosa). **No** congela el contenido. La caja es fija, pero lo de dentro se puede reordenar.

Cuidado: copiar no siempre copia

```
const a = [1, 2, 3];  
const b = a;    // ¡no es una copia! b apunta al MISMO array  
b.push(4);  
console.log(a); // [1, 2, 3, 4] → ¡a también cambió!
```

Con objetos y arrays, `=` copia la **referencia**, no los datos. Para una copia de verdad:

```
const copia = [...a];    // array nuevo independiente  
const copiaObj = { ...persona }; // objeto nuevo independiente
```

Los `...` se llaman *spread* y crean una copia superficial. Este es uno de los conceptos que más errores sutiles evita.

Ejercicio

Crea un array, asígnalo a otra variable con `=`, modifica el segundo y observa que el primero cambia. Luego repítelo usando `[...]` y comprueba que ahora son independientes.

Lección 41 — null y undefined

Ambos significan "ausencia de valor", pero con un matiz importante. Distinguirlos te ayuda a leer mejor lo que pasa en tu programa.

undefined: "no se ha asignado"

Aparece **solo**, sin que tú lo pongas, cuando algo no tiene valor todavía:

```
let x;  
console.log(x);           // undefined → declarada pero sin valor  
  
const obj = {};  
console.log(obj.nombre);  // undefined → propiedad inexistente  
  
function nada() {}  
console.log(nada());      // undefined → función sin return
```

Regla mental: **undefined es de la máquina**. JavaScript lo pone cuando falta algo.

null: "vacío a propósito"

Lo pones **tú** para decir "aquí no hay nada, intencionadamente":

```
let usuario = null;       // todavía no hay usuario logueado  
usuario = { nombre: "Ana" }; // luego sí
```

Regla mental: **null es del programador**. Es una decisión consciente.

Comprobar si hay valor

```
let dato;

if (dato === undefined) { console.log("sin asignar"); }
if (dato === null) { console.log("vacío a propósito"); }

// Cubrir ambos a la vez (lo más práctico):
if (dato == null) { // == aquí sí es útil: pillará null Y undefined
  console.log("no hay valor");
}
```

Este es el **único caso** donde se recomienda `==` en lugar de `===`: `dato == null` detecta tanto `null` como `undefined` de una vez.

El operador ?? (valor por defecto)

Muy útil: da un valor alternativo si algo es `null` o `undefined`:

```
const nombre = usuario?.nombre ?? "Invitado";
```

- `?.` accede de forma segura aunque `usuario` sea `null` (no rompe).
- `??` pone `"Invitado"` si lo de la izquierda no tiene valor.

Ejercicio

Declara una variable sin valor y comprueba que es `undefined`. Luego asígnale `null` y usa `??` "por defecto" para mostrar un texto alternativo.

Lección 42 — Coerción de tipos

La **coerción** es cuando JavaScript **convierte un tipo en otro automáticamente**. Es la causa de los resultados más sorprendentes del lenguaje, así que conviene entenderla para no caer en trampas.

Coerción automática (la traicionera)

```
console.log("5" + 3);    // "53"   → convierte 3 a texto y une
console.log("5" - 3);    // 2       → convierte "5" a número y resta
console.log("5" * "2");  // 10      → ambos a número
console.log(true + 1);   // 2       → true se vuelve 1
```

El **+** es especial: si **un lado es texto**, convierte todo a texto y concatena. El resto de operadores (**-**, *****, **/**) convierten a número. De ahí el desconcierto.

Por qué pasa esto en la vida real

Los datos de formularios e internet llegan como **string**. Si no conviertes, sumas mal:

```
const a = "10";    // de un input
const b = "5";
console.log(a + b); // "105" ← ¡no es 15!
```

Conversión manual (la recomendada)

No dejes que JavaScript adivine: **convierte tú** de forma explícita.


```
console.log(Number("10") + Number("5")); // 15 ✓  
console.log(String(42)); // "42"  
console.log(Boolean(0)); // false  
  
// Atajos comunes:  
console.log(parseInt("10px")); // 10 (extrae el entero)  
console.log(parseFloat("9.99€")); // 9.99  
console.log(+ "42"); // 42 (+ delante convierte a number)
```

Cuidado con NaN

Si conviertes algo que no es número, sale `NaN` (*Not a Number*):

```
console.log(Number("hola")); // NaN  
console.log(isNaN(Number("hola"))); // true → así lo detectas
```

Regla de oro

Convierte siempre de forma explícita. Usa `Number()`, `String()`, `parseInt()` en vez de confiar en la coerción automática. Tu código será predecible y evitarás bugs como `"105"`.

Ejercicio

Tienes `const cantidad = "3"` y `const precio = "9.99"` (ambos texto). Calcula el total **correctamente** convirtiéndolos a número primero.

Lección 43 — Truthy y falsy

En un `if`, JavaScript no necesita un booleano exacto: **cualquier valor** se interpreta como verdadero o falso. A esos valores se les llama *truthy* (cuentan como `true`) y *falsy* (cuentan como `false`).

Los valores falsy (apréndetelos)

Solo hay **unos pocos**. Estos se comportan como `false`:

```
false
0
""      // string vacío
null
undefined
NaN
```

Todo lo demás es truthy, incluido lo que podría engañarte:

```
"0"      // truthy (es un texto, no vacío!)
"false"   // truthy (texto con contenido)
[]        // truthy (array vacío)
{}        // truthy (objeto vacío)
-1        // truthy (cualquier número que no sea 0)
```

Para qué sirve

Permite comprobaciones cortas y naturales:

```
const nombre = "";

if (nombre) {
  console.log("Hola " + nombre);
} else {
  console.log("Falta el nombre"); // se ejecuta: "" es falsy
}
```

En vez de `if (nombre !== "" && nombre !== null)`, basta `if (nombre)`. Cubre "vacío, null o sin definir" de golpe.

Casos prácticos

```
const lista = [];  
if (lista.length) { ... } // 0 es falsy → array vacío = false  
  
function saludar(nombre) {  
  nombre = nombre || "amigo"; // si no hay nombre, usa "amigo"  
  console.log("Hola " + nombre);  
}
```

Cuidado cuando `0` es un valor válido:

```
const stock = 0;  
if (stock) { ... } // ❌ no entra: 0 es falsy, ¡pero 0 es un stock real!  
if (stock !== null) { ... } // ✅ mejor si 0 cuenta
```

Para esos casos, usa el `??` de la Lección 41, que solo reacciona a `null` / `undefined`, no a `0` ni `""`.

Ejercicio

Comprueba en consola si son truthy o falsy: `""`, `"hola"`, `0`, `"0"`, `[]`, `null`. Pista: usa `if (valor) console.log("truthy"); else console.log("falsy");`.

Lección 44 — Errores y try/catch

Los errores son parte normal de programar. Lo importante es **gestionarlos** para que tu programa no se rompa entero cuando algo falla.

Qué pasa cuando hay un error

Por defecto, un error **detiene** el programa en ese punto:

```
const datos = null;
console.log(datos.nombre); // ❌ TypeError: el programa se para aquí
console.log("esto no se ejecuta");
```

try / catch

Permite **intentar** algo y, si falla, **capturar** el error en vez de morir:

```
try {
  const datos = null;
  console.log(datos.nombre); // falla aquí
} catch (error) {
  console.log("Algo salió mal: " + error.message);
}

console.log("el programa sigue vivo"); // ✅ ahora sí se ejecuta
```

- `try { }` → el código que **podría** fallar.
- `catch (error) { }` → qué hacer **si** falla. `error` trae info del fallo.

Cuándo usarlo

No envuelvas todo en `try/catch`. Úsalo donde el fallo es **esperable y externo**: leer datos de internet, procesar lo que escribe un usuario, convertir JSON...

```
function parsearJSON(texto) {
  try {
    return JSON.parse(texto);
  } catch {
    console.log("JSON inválido");
    return null;
  }
}
```

Lanzar tus propios errores

Con `throw` generas un error a propósito cuando algo no debería pasar:

```
function dividir(a, b) {
  if (b === 0) {
    throw new Error("No se puede dividir entre cero");
  }
  return a / b;
}

try {
  dividir(10, 0);
} catch (e) {
  console.log(e.message); // No se puede dividir entre cero
}
```

finally (opcional)

El bloque `finally` se ejecuta **siempre**, falle o no. Útil para limpiar (cerrar conexiones, etc.):

```
try { ... } catch (e) { ... } finally { console.log("terminado"); }
```

Ejercicio

Escribe una función que reciba un texto y lo convierta a número con `Number()`. Si el resultado es `NaN`, **lanza** un error con `throw`. Llámala dentro de un `try/catch`.

Lección 45 — Callbacks

Algunas tareas **tardan**: leer un archivo, pedir datos a internet, esperar unos segundos. No tendría sentido que el programa se congelara esperando. La solución clásica son los **callbacks**: "haz esto y, cuando termines, llama a esta función".

Recordatorio

Un callback es una función que pasas como argumento para que otra la ejecute cuando toque (lo viste en la Lección 22). Aquí lo aplicamos al **tiempo**.

setTimeout: el ejemplo más claro

```
console.log("Antes");

setTimeout(() => {
  console.log("Después de 2 segundos");
}, 2000);

console.log("Después");
```

Salida:

```
Antes
Después
Después de 2 segundos ← llega el último, 2s más tarde
```

`setTimeout` recibe una función (el callback) y un tiempo en milisegundos. El programa **no se para**: sigue con "Después" y ejecuta el callback cuando pasa el tiempo. Esto es la **asincronía**.

Por qué importa

El código no siempre se ejecuta en el orden en que está escrito. Las tareas lentas se "apartan" y su callback corre más tarde. Entender esto es clave para todo lo que tarda.

El problema: callback hell

Cuando una tarea depende de otra, que depende de otra... los callbacks se anidan y el código se vuelve ilegible:

```
pedirUsuario(id, (usuario) => {  
  pedirPedidos(usuario, (pedidos) => {  
    pedirDetalle(pedidos[0], (detalle) => {  
      console.log(detalle); // anidamiento que crece hacia la derecha  
    });  
  });  
});
```

A esto se le llama *callback hell* ("infierno de callbacks"). Es feo y difícil de mantener.

La solución moderna

Para resolver ese problema se inventaron las **promesas** y `async/await`, las dos siguientes lecciones. Pero entender el callback es el primer paso: es la base de todo lo asíncrono.

Ejercicio

Usa `setTimeout` para imprimir `"¡Hola!"` después de 1 segundo. Antes y después del `setTimeout`, pon un `console.log` y observa el orden en que aparecen.

Lección 46 — Promesas

Una **promesa** representa un valor que **aún no está listo** pero que llegará: el resultado de una tarea que tarda. Resuelve el "callback hell" de la lección anterior con una sintaxis más ordenada.

La idea

Piensa en pedir comida con un número de ticket. No tienes la comida aún, pero tienes una **promesa** de que llegará. Una promesa tiene tres estados:

- **pendiente**: aún esperando.
- **cumplida** (*resolved*): salió bien, hay resultado.
- **rechazada** (*rejected*): falló.

Consumir una promesa con `.then`

Cuando una función devuelve una promesa, recoges el resultado con `.then` y los errores con `.catch`:

```
fetch("https://api.ejemplo.com/datos")
  .then(respuesta => respuesta.json()) // cuando llega, procesa
  .then(datos => console.log(datos))   // usa el resultado
  .catch(error => console.log("Error:", error)); // si algo falla
```

`fetch` (pedir datos a internet) devuelve una promesa. Encadenas `.then` para cada paso. Mucho más plano que anidar callbacks.

Por qué es mejor que los callbacks

Compara con el callback hell de la lección anterior:


```
pedirUsuario(id)
  .then(usuario => pedirPedidos(usuario))
  .then(pedidos => pedirDetalle(pedidos[0]))
  .then(detalle => console.log(detalle))
  .catch(error => console.log(error));
```

Se lee de arriba a abajo, en pasos, con **un solo** `.catch` para todos los errores. Mucho más claro.

Crear una promesa (para entenderla)

```
const promesa = new Promise((resolve, reject) => {
  const exito = true;
  if (exito) {
    resolve("¡Listo!");    // sale bien
  } else {
    reject("Algo falló");  // sale mal
  }
});

promesa
  .then(mensaje => console.log(mensaje))
  .catch(error => console.log(error));
```

En la práctica casi siempre **consumirás** promesas que ya te dan (como `fetch`), más que crearlas.

Lo que viene

Las promesas ya son buenas, pero hay una sintaxis todavía más limpia encima de ellas:

`async/await`. Es la siguiente y última pieza del rompecabezas asíncrono.

Ejercicio

Crea una promesa que se resuelva con el texto `"Datos cargados"` después de usar `setTimeout` de 1 segundo dentro de ella. Conéctala con `.then` para mostrar el mensaje.

Lección 47 — async / await

`async/await` es la forma **más moderna y legible** de trabajar con código asíncrono. Hace que el código que tarda **parezca** código normal de arriba a abajo, sin `.then` encadenados.

La idea

Es "azúcar sintáctico" sobre las promesas: por dentro siguen siendo promesas, pero se escriben de forma mucho más natural.

```
async function cargarDatos() {  
  const respuesta = await fetch("https://api.ejemplo.com/datos");  
  const datos = await respuesta.json();  
  console.log(datos);  
}
```

- `async` antes de `function` → marca que la función trabaja con asincronía.
- `await` antes de una promesa → "**espera** a que esté lista y dame el resultado", sin congelar el resto del programa.

Comparación directa

Lo mismo con promesas y con async/await:

```
// Con .then
function cargar() {
  return fetch(url)
    .then(r => r.json())
    .then(datos => console.log(datos));
}

// Con async/await → se lee como pasos normales
async function cargar() {
  const r = await fetch(url);
  const datos = await r.json();
  console.log(datos);
}
```

La segunda versión parece código síncrono de toda la vida. Por eso es la preferida hoy.

Gestionar errores: try/catch

Aquí vuelve el `try/catch` de la Lección 44, que con `async/await` es la forma natural de capturar fallos:

```
async function cargarDatos() {
  try {
    const r = await fetch(url);
    const datos = await r.json();
    console.log(datos);
  } catch (error) {
    console.log("Error al cargar:", error.message);
  }
}
```

Reglas a recordar

1. `await` solo se puede usar **dentro** de una función `async`.
2. Una función `async` siempre devuelve una promesa.
3. Pon `await` delante de **cada** cosa que tarde (cada promesa).

El recorrido asíncrono

Callbacks → promesas → async/await. Cada paso resolvió los problemas del anterior. Hoy escribirás casi todo con async/await, pero entender los tres te permite leer cualquier código.

Ejercicio

Convierte a `async/await` esta función: `function f() { return esperar(1000).then(() => console.log("listo")); }`, donde `esperar` devuelve una promesa. Añádele un `try/catch`.

Lección 48 — Nombrar bien las variables

El mejor código se entiende sin comentarios porque los **nombres** lo explican. Nombrar bien es una de las habilidades más infravaloradas y más valiosas de un programador.

Un nombre debe decir QUÉ guarda

```
// ❌ no dicen nada
let x = 9.99;
let datos = true;
let temp = ["Ana", "Luis"];

// ✅ se entienden solos
let precio = 9.99;
let estaLogueado = true;
let nombresAlumnos = ["Ana", "Luis"];
```

Si tienes que abrir el código y leerlo entero para saber qué contiene una variable, el nombre ha fallado.

Convenciones de JavaScript

- **camelCase** para variables y funciones: `precioTotal`, `calcularIVA`. La primera palabra en minúscula, las siguientes con mayúscula inicial.
- **Constantes "de configuración"** a veces en MAYÚSCULAS: `const IVA = 0.21`.
- **Funciones** suelen empezar por **verbo**: `obtenerUsuario`, `calcularTotal`, `validarEmail`.
- **Booleanos** suelen empezar por `es`, `tiene`, `hay`: `esValido`, `tienePermiso`.

Ni muy corto ni muy largo

```
let n; // ❌ demasiado críptico
let numeroDeUsuariosActivos; // ✅ claro
let elNumeroTotalDeUsuariosQueEstanActivosAhoraMismo; // ❌ excesivo
```

Excepción: en bucles cortos, `i`, `j` están aceptados por convención.

Consistencia

Elige un idioma y un estilo, y mantenlo. No mezcles `userName` con `nombreUsuario` en el mismo proyecto. La coherencia hace el código predecible.

Por qué importa de verdad

Pasarás **más tiempo leyendo** código (el tuyo de hace un mes, el de otros) que escribiéndolo. Buenos nombres convierten esa lectura en algo fluido. Es inversión, no pérdida de tiempo.

Ejercicio

Mejora estos nombres: `let d = 30;` (días de un mes), `function calc(a, b)` (suma dos precios), `let flag = false;` (indica si el pedido está pagado).

Lección 49 — Depurar (debugging)

Depurar es encontrar y arreglar errores (*bugs*). No es algo que les pase a los malos programadores: es el **80 % del trabajo** de cualquiera. Saber depurar bien marca la diferencia.

El primer reflejo: console.log

La herramienta más simple y más usada. Imprime valores para ver qué está pasando de verdad:

```
function calcularTotal(precio, cantidad) {  
  console.log("precio:", precio, "cantidad:", cantidad); // ¿qué llega?  
  const total = precio * cantidad;  
  console.log("total:", total); // ¿qué sale?  
  return total;  
}
```

Cuando algo no funciona, **no adivines**: imprime los valores en cada paso y compáralos con lo que esperabas. El error casi siempre está donde la realidad y tu expectativa difieren.

Leer el mensaje de error

Los errores no son tus enemigos: te **dicen** qué pasó y **dónde**.

```
TypeError: Cannot read properties of null (reading 'nombre')  
    at calcular (script.js:14)
```

- **Tipo:** `TypeError` (intentaste algo imposible para ese tipo).
- **Causa:** leer `.nombre` de algo que es `null`.
- **Dónde:** línea 14.

Lee el mensaje entero antes de tocar nada. La mitad de los bugs se resuelven solo con leerlo bien.

Errores comunes al empezar

- `is not defined` → variable mal escrita o fuera de su scope (Lección 20).
- `is not a function` → llamaste algo que no es función, o un typo en el método.
- `Cannot read properties of undefined` → accediste a algo que no existe.
- `NaN` en cálculos → mezclaste texto y número (Lección 42).

Método para depurar

1. **Reproduce** el error de forma fiable.
2. **Localiza** la línea (el mensaje te la da).
3. **Imprime** los valores justo antes con `console.log`.
4. **Compara** lo que hay con lo que esperabas.
5. **Cambia una cosa** y vuelve a probar. No diez a la vez.

El debugger del navegador

Más potente que `console.log` : en las herramientas (F12), pestaña *Sources*, puedes poner *breakpoints* (pausas) y ejecutar línea a línea viendo cada variable. Cuando `console.log` se quede corto, este es el siguiente nivel.

Ejercicio

Este código falla. Encuentra por qué usando `console.log` :

```
function media(numeros) {  
  let suma = 0;  
  for (let i = 0; i <= numeros.length; i++) {  
    suma += numeros[i];  
  }  
  return suma / numeros.length;  
}  
console.log(media([10, 20, 30])); // da NaN, ¿por qué?
```

Pista: mira con atención la condición del bucle.

Lección 50 — Pensamiento computacional

Llegaste al final. Esta lección no es sintaxis: es la **mentalidad** que conecta las otras 49. Programar bien no es saber muchos comandos, sino saber **pensar problemas** de cierta forma.

Las cuatro habilidades clave

1. Descomposición

Partir un problema grande en pasos pequeños y resolubles. "Hacer una tienda online" es enorme; "mostrar un producto", "sumar el carrito", "validar el pago" son abordables. Ya lo hiciste con la receta del café de la Lección 1.

2. Reconocer patrones

Ver lo que se repite. Si tres trozos de código hacen casi lo mismo, ahí hay una **función** (Lección 17). Si recorres una lista una y otra vez, ahí hay un **bucle** o un **map**. Los patrones te dicen qué herramienta usar.

3. Abstracción

Quedarte con lo esencial e ignorar el detalle. No necesitas saber cómo funciona `map` por dentro para usarlo. Creas funciones con buen nombre y luego las usas sin pensar en su interior. Así construyes cosas complejas con piezas simples.

4. Algoritmos

Diseñar la secuencia exacta de pasos. Antes de escribir código, piensa la lógica: ¿qué datos entran, qué pasos los transforman, qué sale?

El método que de verdad funciona

1. **Entiende** el problema. Escríbelo con tus palabras. La mitad de los fallos vienen de resolver el problema equivocado.
2. **Planifica** en pseudocódigo o en español, sin sintaxis:

- recorrer la lista de productos
- quedarme con los que cuestan menos de 20€
- sumar sus precios
- mostrar el total

3. **Traduce** cada paso a código (aquí entran filter, reduce...).
4. **Prueba** con casos pequeños.
5. **Mejora** los nombres y simplifica.

Cómo seguir mejorando

- **Programa cada día**, aunque sean 15 minutos. La constancia gana a las maratones.
- **Construye proyectos pequeños** que te interesen: una calculadora, una lista de tareas, un conversor. Aprenderás más que con mil tutoriales.
- **Equivócate mucho**. Cada bug resuelto te enseña algo que no se olvida.
- **Lee código de otros**. Verás patrones y trucos nuevos.

Lo que llevas

Con estos 50 conceptos tienes los **fundamentos reales** de la programación. No son exclusivos de JavaScript: variables, condicionales, bucles, funciones, estructuras de datos y esta forma de pensar existen en **todos** los lenguajes. Cambiar a Python, Java o cualquier otro será solo aprender sintaxis nueva sobre ideas que ya dominas.

Enhorabuena. Ya sabes pensar como un programador. Ahora ve a construir algo.

Ejercicio final

Elige un problema cotidiano (organizar una lista de la compra, calcular gastos del mes...) y resuélvelo: primero descríbelo, luego escríbelo en pseudocódigo, y por último prográmalo en JavaScript usando lo aprendido en el curso.



Glosario de términos

Los términos que más aparecen en el curso, explicados en una línea.

Término	Significado
Algoritmo	Secuencia de pasos para resolver un problema.
Argumento	Valor real que pasas a una función al llamarla.
Array	Lista ordenada de valores. Se escribe con <code>[]</code> .
Asíncrono	Código que no se ejecuta de inmediato; espera a que algo termine.
Booleano	Tipo de dato con solo dos valores: <code>true</code> o <code>false</code> .
Bucle	Estructura que repite código (<code>for</code> , <code>while</code>).
Bug	Error en el código.
Callback	Función que se pasa a otra para que la ejecute más tarde.
camelCase	Estilo de nombres: <code>precioTotal</code> .
Coerción	Conversión automática de un tipo a otro.
Concatenar	Unir textos con <code>+</code> .
Condicional	Estructura que decide (<code>if</code> , <code>else</code>).
const	Declara una variable que no se puede reasignar.
Depurar (debug)	Buscar y arreglar errores.
Desestructuración	Extraer valores de objetos/arrays a variables.
Falsy	Valor que cuenta como <code>false</code> (<code>0</code> , <code>""</code> , <code>null</code> ...).
Función	Bloque de código reutilizable con nombre.
Índice	Posición de un elemento en un array (empieza en 0).
Inmutable	Que no se puede cambiar después de crearse.

Término	Significado
let	Declara una variable que sí se puede reasignar.
Método	Función que pertenece a un objeto (<code>array.push()</code>).
Mutar	Modificar algo después de crearlo.
NaN	"Not a Number": resultado de un cálculo numérico inválido.
null	Ausencia de valor puesta a propósito.
Objeto	Colección de parejas clave-valor. Se escribe con <code>{ }</code> .
Operador	Símbolo que opera con datos (<code>+</code> , <code>===</code> , <code>&&</code>).
Parámetro	Variable que recibe una función en su declaración.
Primitivo	Tipo simple: number, string, boolean, null, undefined.
Promesa	Objeto que representa un valor futuro de una tarea asíncrona.
Recursión	Función que se llama a sí misma.
return	Devuelve un valor desde una función.
Scope (ámbito)	Zona donde una variable existe y es accesible.
String	Texto. Entre comillas.
Template literal	Texto con backticks que admite <code>\${variables}</code> .
Truthy	Valor que cuenta como <code>true</code> (casi todo lo no-falsy).
typeof	Operador que devuelve el tipo de un valor.
undefined	Ausencia de valor puesta por la máquina.
Variable	Caja con nombre que guarda un dato.

Chuleta de sintaxis JavaScript

Referencia rápida de todo lo visto en el curso. Tenla a mano mientras programas.

Variables

```
const nombre = "Ana"; // no se reasigna (por defecto)
let edad = 25;        // se puede reasignar
```

Tipos

```
42           // number
"texto"      // string
true / false // boolean
null         // vacío a propósito
undefined    // sin asignar
typeof valor // saber el tipo
```

Operadores

```
+ - * / %      // aritméticos (% = resto)
+= -= *=      // asignación corta
++ --         // incremento / decremento
=== !==       // comparación (usa SIEMPRE 3 iguales)
> < >= <=     // comparación
&& || !       // lógicos: Y, O, NO
```

Condicionales

```
if (cond) {  
  ...  
} else if (otra) {  
  ...  
} else {  
  ...  
}  
  
const x = cond ? siVerdadero : siFalso;    // ternario  
  
switch (valor) {  
  case "a": ...; break;  
  default: ...;  
}
```

Bucles

```
for (let i = 0; i < 5; i++) { ... }  
while (cond) { ... }  
for (const item of array) { ... }  
for (const clave in objeto) { ... }  
break;      // sale del bucle  
continue;   // salta a la siguiente vuelta
```

Funciones

```
function sumar(a, b) { return a + b; }  
const sumar = (a, b) => a + b;           // arrow  
const doble = n => n * 2;                // un parámetro  
function f(x = 10) { ... }              // valor por defecto
```

Arrays

```

const a = [1, 2, 3];
a.length           // tamaño
a[0]               // primer elemento
a.push(4) a.pop()  // final: añadir / quitar
a.unshift(0) a.shift() // inicio: añadir / quitar
a.includes(2)      // ¿está?
a.indexOf(2)       // posición o -1
a.join("-")        // a texto

a.map(n => n * 2)    // transformar → array nuevo
a.filter(n => n > 1) // seleccionar → array nuevo
a.reduce((acc, n) => acc + n, 0) // combinar → un valor
a.forEach(n => console.log(n))  // acción por elemento

```

Objetos

```

const obj = { nombre: "Ana", edad: 25 };
obj.nombre           // leer (punto)
obj["edad"]          // leer (corchetes)
obj.ciudad = "Madrid" // añadir / cambiar
delete obj.edad       // borrar

const persona = {
  nombre: "Ana",
  saludar() { return "Soy " + this.nombre; }
};

const { nombre, edad } = obj; // desestructurar

```

Strings

```

texto.length
texto.toUpperCase() texto.toLowerCase()
texto.trim()
texto.includes("x") texto.startsWith("x") texto.endsWith("x")
texto.replace("a", "b") texto.replaceAll("a", "b")
texto.split(",") // a array
array.join(",") // a texto
`Hola ${nombre}` // template literal

```

Conversión de tipos

```
Number("42")      // a número
String(42)         // a texto
Boolean(0)         // a booleano
parseInt("10px")   // entero
parseFloat("9.9")  // decimal
```

Errores

```
try {
  ...
} catch (error) {
  console.log(error.message);
} finally {
  ...
}
throw new Error("mensaje");
```

Asíncrono

```
setTimeout(() => { ... }, 1000); // tras 1s

promesa.then(r => ...).catch(e => ...);

async function f() {
  try {
    const r = await fetch(url);
    const datos = await r.json();
  } catch (e) { ... }
}
```

Útiles

```
console.log(...)           // imprimir
const copia = [...array]   // copiar array
const copia = { ...obj }   // copiar objeto
valor ?? "defecto"         // si es null/undefined
obj?.propiedad             // acceso seguro
```

Terminaste los 50 conceptos. Ahora toca convertir conocimiento en habilidad real. Esta es la ruta.

1. Construye proyectos pequeños (lo más importante)

Aprender de verdad pasa **construyendo**, no leyendo. Ideas para empezar, de menos a más:

- **Calculadora de propinas:** pides cuenta y porcentaje, muestras total. (variables, operadores)
- **Conversor de unidades:** euros↔dólares, °C↔°F. (funciones)
- **Lista de la compra:** añadir, quitar, marcar comprado. (arrays, objetos)
- **Generador de contraseñas:** longitud y tipos a elegir. (strings, bucles)
- **To-do list en una web:** tu primer proyecto con HTML real.
- **Quiz / test de preguntas:** array de objetos con preguntas y respuestas.

Regla de oro: si un proyecto te aburre, cámbialo por uno que te ilusione. La motivación es el mejor profesor.

2. Lleva JavaScript a una página web

Hasta ahora usaste la consola. El siguiente salto natural es manipular una **página web** con JavaScript (lo que se llama el *DOM*):

- Aprende lo básico de **HTML** y **CSS** (un par de tardes).
- Conecta tu JS con botones, formularios y textos de la página.
- Busca: "*JavaScript DOM para principiantes*".

Ahí es donde tu código empieza a **verse y tocarse**, y engancha de verdad.

3. Profundiza en JavaScript

Conceptos del siguiente nivel, cuando te sientas cómodo:

- El **DOM** y los eventos (clics, teclado).
- **fetch** y APIs: traer datos reales de internet.
- **Módulos** (`import` / `export`): organizar el código en archivos.
- **Clases:** otra forma de crear objetos.

4. ¿Otro lenguaje?

Recuerda la Lección 50: los **fundamentos son universales**. Si quieres explorar:

- **Python** → ciencia de datos, IA, automatización, scripts. Sintaxis muy amable.
- **JavaScript (sigue)** → todo lo web, apps, servidores (Node.js).
- **SQL** → bases de datos. Imprescindible casi en cualquier trabajo.

No saltes de lenguaje en lenguaje sin terminar nada. Domina uno y los demás caerán solos.

5. Hábitos que marcan la diferencia

- **Programa casi a diario**. 20 min constantes > 5 horas un domingo al mes.
- **Aprende a buscar**. Saber googlear bien un error es una superhabilidad real.
- **Lee la documentación**. MDN (developer.mozilla.org) es la biblia de JavaScript, gratis.
- **No copies sin entender**. Si pegas código, léelo hasta saber qué hace cada línea.
- **Únete a una comunidad**. Foros, Discord, grupos locales. Programar acompañado motiva.

Recursos gratuitos recomendados

- **MDN Web Docs** — referencia oficial de JavaScript.
- **freeCodeCamp** — ejercicios prácticos gratis.
- **The Odin Project** — ruta completa de desarrollo web.
- **Exercism / Codewars** — retos de código para practicar.

Un último consejo

Nadie nace sabiendo programar. Todo programador que admiras estuvo donde estás tú hoy, frente a un error que no entendía. La diferencia no es talento: es **no rendirse y seguir construyendo**.

Ya tienes los cimientos. El resto se construye encima, una línea de código cada día.

¡A por ello! 🚀